

// Jivops

Guía completa — 30 lecciones

Imágenes y builds · Redes · Seguridad · Compose avanzado ·
Observabilidad · Operación

Contenido

Módulo 1: Imágenes y builds

- Lección 01 — Multi-stage builds
- Lección 02 — Optimización de capas y layer caching
- Lección 03 — .dockerignore y contexto de build
- Lección 04 — Imágenes base — slim, alpine, distroless
- Lección 05 — BuildKit y build avanzado

Módulo 2: Redes y almacenamiento

- Lección 06 — Redes Docker: bridge, host, overlay
- Lección 07 — Docker volumes vs bind mounts
- Lección 08 — Persistencia de datos en contenedores
- Lección 09 — DNS interno y comunicación entre contenedores
- Lección 10 — Docker networks personalizadas

Módulo 3: Seguridad

- Lección 11 — Escaneo de vulnerabilidades (Trivy, Docker Scout)
- Lección 12 — Ejecutar contenedores sin root
- Lección 13 — Capabilities y seccomp
- Lección 14 — Gestión de secretos en Docker
- Lección 15 — Firma y verificación de imágenes (Docker Content Trust)

Módulo 4: Compose avanzado

- Lección 16 — Docker Compose — servicios avanzados
- Lección 17 — Healthchecks en Docker Compose
- Lección 18 — Docker Compose para múltiples entornos
- Lección 19 — Restart policies y resiliencia
- Lección 20 — Resource limits en Docker (memory, cpus)

Módulo 5: Observabilidad

- Lección 21 — Logging de contenedores (drivers de log)
- Lección 22 — Monitoreo de contenedores con cAdvisor/Prometheus
- Lección 23 — Debugging de contenedores en producción
- Lección 24 — Docker exec y troubleshooting en caliente
- Lección 25 — Optimización de rendimiento de contenedores

Módulo 6: Operación

Lección 26 — Registries privados (Docker Registry, Harbor)

Lección 27 — CI/CD para builds de imágenes Docker

Lección 28 — Actualización y rollback de contenedores

Lección 29 — Docker en producción: buenas prácticas de despliegue

Lección 30 — Proyecto final: aplicación containerizada lista para producción

Módulo 1: Imágenes y builds

Lección 01 — Multi-stage builds

Un multi-stage build deja la imagen final sin herramientas de compilación innecesarias, copiando solo el binario compilado desde la etapa 'builder'.

```
FROM node:20 AS builder
```

Lección 02 — Optimización de capas y layer caching

Copiar package.json antes que el resto del código permite a Docker reutilizar la capa de dependencias si ese archivo no cambió, acelerando builds futuros.

```
COPY package.json ./
```

Lección 03 — .dockerignore y contexto de build

Un .dockerignore bien configurado evita enviar archivos innecesarios (como .git o node_modules) al contexto de build, que no aportan nada al runtime.

```
node_modules
```

Lección 04 — Imágenes base — slim, alpine, distroless

Una imagen distroless no incluye shell ni gestor de paquetes, reduciendo la superficie de ataque: un atacante no podría ejecutar comandos aunque comprometa el proceso.

Lección 05 — BuildKit y build avanzado

BuildKit permite builds paralelos más eficientes: etapas independientes se construyen simultáneamente en vez de esperar una a otra en secuencia.

```
DOCKER_BUILDKIT=1
```

Módulo 2: Redes y almacenamiento

Lección 06 — Redes Docker: bridge, host, overlay

La red 'overlay' conecta contenedores en distintos hosts físicos (Swarm). La red 'host' elimina el aislamiento de red, compartiendo directamente la interfaz del anfitrión.

Lección 07 — Docker volumes vs bind mounts

Un volume es gestionado por Docker y más portable; un bind mount apunta a una ruta específica del host. Para datos de base de datos suele preferirse el volume.

```
docker run -v mivolumen:/data
```

Lección 08 — Persistencia de datos en contenedores

Sin un volume o bind mount, los datos escritos dentro de un contenedor se pierden irrecuperablemente al eliminarlo — un error común y grave en producción.

Lección 09 — DNS interno y comunicación entre contenedores

En una red Docker personalizada, los contenedores se comunican por nombre gracias al DNS interno de Docker — la red bridge por defecto no ofrece esta resolución automática.

```
docker network create mired
```

Lección 10 — Docker networks personalizadas

Solo los contenedores conectados a una red específica pueden comunicarse entre sí directamente, útil para aislar por ejemplo frontend y backend.

Módulo 3: Seguridad

Lección 11 — Escaneo de vulnerabilidades (Trivy, Docker Scout)

Trivy revisa vulnerabilidades conocidas en el sistema operativo base y las dependencias de la imagen, idealmente integrado en el pipeline CI/CD antes de llegar a producción.

```
trivy image miapp:1.0
```

Lección 12 — Ejecutar contenedores sin root

Ejecutar con un usuario no privilegiado (USER appuser) limita lo que un atacante podría hacer si compromete el proceso, aplicando el principio de menor privilegio.

```
USER appuser
```

Lección 13 — Capabilities y seccomp

--cap-drop=ALL elimina todas las capabilities del kernel por defecto, añadiendo solo las estrictamente necesarias para reducir la superficie de ataque.

```
docker run --cap-drop=ALL
```

Lección 14 — Gestión de secretos en Docker

Pasar contraseñas como ENV en el Dockerfile las deja visibles en el historial de capas. Un mecanismo dedicado (como docker secret en Swarm) evita esa exposición.

Lección 15 — Firma y verificación de imágenes (Docker Content Trust)

Docker Content Trust garantiza que una imagen no fue alterada y proviene realmente del publicador esperado, protegiendo contra ataques de supply chain.

```
DOCKER_CONTENT_TRUST=1
```

Módulo 4: Compose avanzado

Lección 16 — Docker Compose — servicios avanzados

`depends_on` define orden de arranque entre servicios, pero solo espera a que el contenedor inicie, no a que el servicio dentro esté realmente listo.

Lección 17 — Healthchecks en Docker Compose

Un healthcheck verifica si el servicio realmente funciona, no solo si el proceso arrancó. Combinarlo con `condition: service_healthy` es más confiable que `depends_on` simple.

Lección 18 — Docker Compose para múltiples entornos

Los archivos de override (-f `docker-compose.prod.yml`) mantienen una base común compartida entre entornos, declarando solo las diferencias específicas de cada uno.

Lección 19 — Restart policies y resiliencia

'unless-stopped' reinicia el contenedor automáticamente ante fallos, recuperando el servicio sin intervención manual.

```
restart: unless-stopped
```

Lección 20 — Resource limits en Docker (memory, cpus)

Limitar memoria con `--memory` evita que un contenedor con un bug (memory leak) consuma todos los recursos del host y afecte a otros servicios.

```
docker run --memory=512m
```

Módulo 5: Observabilidad

Lección 21 — Logging de contenedores (drivers de log)

El logging driver controla a dónde se envían los logs. Enviarlos a un sistema externo hace que sobrevivan aunque el contenedor se elimine, y quedan centralizados.

Lección 22 — Monitoreo de contenedores con cAdvisor/Prometheus

cAdvisor expone métricas de CPU, memoria, red y disco por contenedor en formato Prometheus, permitiendo identificar exactamente qué contenedor causa un problema.

Lección 23 — Debugging de contenedores en producción

Los logs recientes y el exit code son el primer lugar a revisar al depurar un contenedor que se reinicia constantemente.

```
docker logs --tail 100 nombre_contenedor
```

Lección 24 — Docker exec y troubleshooting en caliente

docker exec abre una shell dentro de un contenedor ya en ejecución, permitiendo investigar el problema en el mismo entorno real sin reproducirlo aparte.

```
docker exec -it micontenedor bash
```

Lección 25 — Optimización de rendimiento de contenedores

Una imagen más pequeña, con menos capas y dependencias, no solo ahorra espacio: también acelera la descarga y el despliegue en producción.

Módulo 6: Operación

Lección 26 — Registries privados (Docker Registry, Harbor)

Un registry privado da control de acceso y privacidad sobre imágenes propietarias, evitando exponer código sensible en un registro público.

Lección 27 — CI/CD para builds de imágenes Docker

Un pipeline típico construye, escanea y versiona la imagen antes de publicarla. Versionar con tags específicos (no solo 'latest') facilita hacer rollback.

```
docker build -t app:1.0 .
```

Lección 28 — Actualización y rollback de contenedores

Tener versiones etiquetadas permite volver rápido a una versión anterior conocida. Desplegar siempre con 'latest' complica saber a qué versión estás volviendo.

Lección 29 — Docker en producción: buenas prácticas de despliegue

Definir resource limits, healthchecks y restart policies juntos protege recursos, detecta fallos reales y recupera el servicio automáticamente.

Lección 30 — Proyecto final: aplicación containerizada lista para producción

Se integra todo lo anterior: multi-stage build, imagen mínima, escaneo de seguridad, healthchecks, resource limits y logging centralizado — el nivel esperado de una app containerizada lista para producción real.